

Data Poisoning y Seguridad del Modelo

Módulo 12 · Seguridad en IA

Tipos de poisoning, backdoor attacks, PoisonGPT, supply chain con safetensors y ModelScan.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

Data Poisoning y Seguridad del Modelo

El data poisoning y los ataques al modelo son la categoría de vulnerabilidades LLM que operan en las capas más profundas del sistema: no en el prompt (que es temporal) sino en los datos de entrenamiento y en los pesos del modelo (que son permanentes). Un ataque de data poisoning exitoso contamina el modelo de forma que puede no ser detectable en pruebas estándar pero se activa ante inputs específicos (backdoor attacks) o produce comportamientos sesgados de forma sistemática.

El OWASP LLM04:2025 (Data and Model Poisoning) y el OWASP LLM03:2025 (Supply Chain) cubren esta categoría. Los repositorios de modelos abiertos (Hugging Face) han emergido como un nuevo vector de ataque: JFrog reportó en 2024 que investigadores identificaron modelos en Hugging Face con backdoors silenciosos que se ejecutaban cuando el modelo era cargado a través del formato pickle. "ML Model Repositories: The Next Big Supply Chain Attack Target" (OffSecML) identifica esto como una amenaza creciente para 2025-2026.

Los tipos de ataques al modelo

Data poisoning en preentrenamiento

Los LLMs se preentrenan sobre datasets masivos extraídos de internet (Common Crawl, Wikipedia, libros, etc.). Un atacante que puede introducir contenido en esas fuentes públicas durante el periodo de recolección de datos puede influir en el comportamiento del modelo. La escala necesaria para afectar un LLM de 70B parámetros es muy grande, pero ataques específicos de nicho (contaminar fuentes de un dominio específico) pueden producir resultados concretos. PoisonGPT (Mithril Security) demostró cómo introducir información falsa de forma quirúrgica en un LLM y publicarlo en Hugging Face como una versión "limpia" del modelo.

Data poisoning en fine-tuning

El fine-tuning es más vulnerable al poisoning que el preentrenamiento porque usa datasets mucho más pequeños: contaminar el 1% de un dataset de fine-tuning de 1.000 ejemplos requiere solo 10 ejemplos maliciosos. Los vectores de ataque son: datasets descargados de fuentes públicas sin verificación de origen, datasets generados con LLMs que pueden haberse propagado con sesgo o información falsa, y datasets producidos por anotadores externos (crowdsourcing) que pueden contener anotaciones maliciosas. La guía del OWASP recomienda usar DVC (Data Version Control) para trackear cambios en los datasets.

Backdoor attacks: activadores ocultos

Los backdoor attacks introducen un "trigger" (una frase, un token, o un patrón específico) que activa un comportamiento especial del modelo. El modelo funciona normalmente ante inputs estándar, pero cuando recibe el trigger, cambia su comportamiento (revelar información confidencial, producir outputs incorrectos específicos, ejecutar acciones no autorizadas). BadRAG (Xue et al., 2024) demostró que passages adversarialmente envenenados pueden

actuar como backdoor semánticos en sistemas RAG, activando comportamientos específicos cuando esos passages son recuperados.

SECCIÓN 3 · CÓMO FUNCIONA

Mitigaciones para data poisoning y supply chain

Para el pipeline de fine-tuning

Prácticas de seguridad para el pipeline de fine-tuning: verificar la procedencia de todos los datasets (¿quién los creó?, ¿cuándo?, ¿se han verificado contra un golden set?). Usar DVC (Data Version Control) para trackear el historial de cambios en los datasets: cualquier modificación genera un hash diferente que puede detectarse. Implementar detección de anomalías en el dataset (identificar ejemplos estadísticamente inusuales que podrían ser envenenados). Separar claramente los datos de entrenamiento de los datos de evaluación y nunca usar los datos de evaluación para entrenamiento.

Para la gestión de modelos base y repositorios

Verificar siempre el hash del modelo descargado con el hash publicado por la fuente oficial. Usar exclusivamente el formato safetensors para modelos descargados de repositorios públicos: los safetensors no permiten la ejecución de código arbitrario durante la carga, eliminando el vector de ataque de malware embebido. Evaluar los modelos descargados en un entorno sandbox aislado antes de usarlos en producción. Para modelos en producción, implementar monitorización de comportamiento que detecte cambios estadísticamente significativos en el output del modelo que podrían indicar activación de un backdoor.

Detección de comportamientos anómalos post-despliegue

Monitorizar las distribuciones de output del modelo a lo largo del tiempo para detectar derive anómalo: si el modelo empieza a producir outputs con una distribución estadísticamente diferente a la baseline, puede indicar que está siendo activado por un trigger de backdoor o que los datos del corpus RAG han sido envenenados. Las métricas de monitorización incluyen: distribución de tokens de output, tasa de respuestas fuera del dominio, y frecuencia de referencias a temas específicos no relacionados con las consultas del usuario.

SECCIÓN 4 · EJEMPLOS REALES

- PoisonGPT (Mithril Security, 2023): demostración de data poisoning en fine-tuning. Los investigadores modificaron quirúrgicamente los pesos de un modelo de Llama para que afirmara que el primer humano en el espacio era Yuri Gagarin pero el 4 de julio (fecha incorrecta). El modelo respondía correctamente a todas las otras preguntas, pasaba todos los benchmarks estándar, y solo mostraba el comportamiento malicioso ante la pregunta específica. Publicaron el modelo en Hugging Face para demostrar que el supply chain de modelos es un vector de ataque real.
- TrojanRAG (Cheng et al., 2024): investigadores demostraron que passages adversarialmente diseñados podían actuar como backdoors semánticos en sistemas RAG. Cuando el sistema RAG recuperaba el passage envenenado (porque era relevante para la consulta), el LLM activaba el comportamiento backdoor aunque el passage envenenado no

contuviera instrucciones explícitas sino patrones semánticos específicos.

- Modelos con malware en pickle (JFrog, 2024): investigadores de seguridad encontraron en Hugging Face decenas de modelos en formato pickle que contenían código malicioso embebido. Al cargar el modelo, el código se ejecutaba silenciosamente, pudiendo robar credenciales, instalar backdoors, o exfiltrar datos. Los modelos tenían nombres similares a modelos legítimos populares (typosquatting).

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Asumir que los modelos de repositorios públicos han sido verificados. Hugging Face y otros repositorios tienen mecanismos de detección pero no son perfectos. La responsabilidad de verificar la seguridad del modelo recae en el consumidor. Usa siempre safetensors, verifica hashes, y evalúa en sandbox antes de producción.

Error 2: No versionar los datasets de fine-tuning. Sin versionado, es imposible saber cuándo se introdujeron datos maliciosos en el pipeline de fine-tuning o qué versión del dataset produjo qué comportamiento en el modelo. DVC o cualquier sistema de control de versiones de datos es imprescindible.

Error 3: Confiar en benchmarks estándar para detectar backdoors. Los backdoors están diseñados para pasar los benchmarks estándar. Solo se activan ante el trigger específico. La detección de backdoors requiere testing específico de comportamiento adversarial, no solo evaluación de calidad estándar.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

Checklist de seguridad del pipeline de modelos: usar safetensors para todos los modelos descargados. Verificar el hash SHA-256 del modelo contra el publicado por el autor. Evaluar el modelo en sandbox antes de despliegue a producción. Versionar todos los datasets de fine-tuning con DVC o git-lfs. Auditar el origen de los datos de fine-tuning. Implementar detección de anomalías en el output del modelo en producción. Documentar la cadena de custodia de cada modelo (quién lo creó, cuándo, con qué datos).

Para la detección de modelos comprometidos: herramienta ProtectAI ModelScan (open source, Python): escanea modelos en formato pickle, H5, y safetensors para detectar código malicioso y serialización peligrosa. Instalación: `pip install modelscan`. Uso: `modelscan -p ruta/al/modelo`. Detecta código embebido en pickles, llamadas a funciones peligrosas, y patrones de serialización conocidos como vectores de ataque.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M4-T6 (Fine-tuning): el pipeline de fine-tuning es el principal vector de data poisoning.
- M8-T8 (Versionado de modelos): el versionado de modelos y datasets es la primera defensa contra data poisoning.
- M12-T1 (OWASP Top 10): data poisoning es LLM04 y supply chain es LLM03 del OWASP LLM 2025.

SECCIÓN 8 · RESUMEN EJECUTIVO

Data poisoning contamina el modelo de forma permanente a través de los datos de entrenamiento. Los tipos son: poisoning en preentrenamiento (escala masiva), poisoning en fine-tuning (más vulnerable, 1% de los datos es suficiente), y backdoor attacks (triggers ocultos que activan comportamientos específicos). Supply chain risks: modelos con malware embebido en pickle (JFrog, 2024), typosquatting en Hugging Face. Mitigaciones: usar safetensors (no pickle), verificar hashes SHA-256, evaluar en sandbox antes de producción, versionar datasets con DVC, ModelScan de ProtectAI para escaneo de modelos. Los backdoors no se detectan con benchmarks estándar: requieren testing de comportamiento adversarial específico.